ATIVIDADE SOBRE COESÃO - RELATÓRIO TÉCNICO V2

Projeto em C com módulos de coesão comunicacional, procedural, temporal, lógica e coincidental

Este relatório apresenta a versão 2 do projeto acadêmico em linguagem C. A nova versão substitui totalmente os módulos anteriores e passa a demonstrar apenas os cinco tipos de coesão solicitados na atividade: comunicacional, procedural, temporal, lógica e coincidental. A proposta foi manter o código organizado, autoexplicativo e fácil de defender em sala.

Versão analisada	V2 do projeto
Linguagem	C
Foco didático	Explicar os novos tipos de coesão solicitados
Estilo adotado	Clean Code, nomes claros e estrutura modular

1. Objetivo da versão 2

A V2 foi criada para alinhar o projeto exatamente ao enunciado atualizado da atividade. Na versão anterior, o sistema demonstrava coesão funcional e sequencial. Nesta nova entrega, esses exemplos foram removidos e deram lugar a cinco módulos novos, cada um pensado para representar um tipo de coesão específico pedido pelo professor.

- substituir os módulos antigos por módulos coerentes com o novo enunciado;
- manter uma organização didática, simples de compilar e simples de apresentar;
- preservar a ideia de código limpo, com nomes legíveis e responsabilidades bem definidas;
- deixar o main.c enxuto, servindo apenas como ponto de demonstração dos módulos.

2. Principais mudanças em relação à V1

Aspecto	V1	V2
Tipos de coesão	funcional e sequencial	comunicacional, procedural, temporal, lógica e coincidental
Modelagem	aluno e pedido	funcionária, ticket de serviço, inicialização, notificação e utilitários diversos
Objetivo didático	mostrar dois tipos clássicos	mostrar exatamente os cinco tipos pedidos no novo enunciado
main.c	demonstrava 2 módulos	demonstrava 5 módulos, mantendo leitura direta
Explicação oral	focada em média do aluno e pipeline de pedido	focada em cinco cenários independentes e fáceis de justificar

Em termos acadêmicos, a mudança mais importante foi a troca completa da base conceitual do projeto. A V2 não é apenas uma expansão da V1; ela é uma reestruturação guiada pelo novo escopo da atividade.

3. Organização do projeto na V2

A estrutura continuou modular. Os arquivos de cabeçalho ficaram em include/ e as implementações em src/. O projeto também preservou um main.c pequeno e focado apenas na demonstração dos módulos.

- include/models/employee.h, service_ticket.h, startup_context.h e notification.h: estruturas de dados do domínio;
- include/modules/...: contratos dos módulos específicos de coesão;
- src/modules/...: implementação dos cinco exemplos pedidos;
- src/main.c: coordenação da demonstração completa do programa.

4. Explicação dos módulos da nova versão

4.1 Coesão comunicacional - employee_record.c

A coesão comunicacional ocorre quando as funções do módulo trabalham sobre o mesmo conjunto de dados. Neste caso, todas as rotinas giram em torno do registro Employee. O módulo calcula compensação mensal, tempo de empresa, assiduidade e elegibilidade para promoção. Mesmo com tarefas diferentes, todas usam o mesmo registro central, o que caracteriza esse tipo de coesão.

4.2 Coesão procedural - service_workflow.c

A coesão procedural aparece quando as funções precisam obedecer a uma ordem de execução. No módulo de atendimento, o fluxo define prioridade, estima horas e fecha a decisão final do procedimento. As etapas não representam necessariamente transformação de uma mesma estrutura em pipeline clássico, mas fazem sentido porque são executadas em uma sequência lógica de trabalho.

4.3 Coesão temporal - startup_tasks.c

A coesão temporal foi representada por tarefas de inicialização. Carregar configuração, iniciar logger, conectar banco e aquecer cache são ações que pertencem ao mesmo momento do ciclo de vida da aplicação: o instante de subida do sistema. O vínculo entre elas não é o mesmo dado nem a mesma lógica decisória, mas sim o fato de acontecerem juntas no startup.

4.4 Coesão lógica - notification_dispatcher.c

A coesão lógica acontece quando várias funções pertencem à mesma categoria conceitual e a escolha entre elas depende de uma condição. No dispatcher de notificações, o sistema decide entre envio por e-mail, SMS ou push. Todas são alternativas do mesmo tipo de serviço, o que justifica a classificação como coesão lógica.

4.5 Coesão coincidental - misc_helpers.c

A coesão coincidental foi usada como contraexemplo didático. O módulo junta funções sem relação direta entre si: cálculo de área do círculo, contagem de vogais, teste de número par e impressão de banner. Esse agrupamento existe apenas por conveniência, não por responsabilidade comum. Justamente por isso ele ajuda a explicar por que esse tipo de coesão é considerado fraco.

5. Como o main.c foi mantido limpo

Assim como na V1, a entrada principal do programa não concentra regra de negócio. O main apenas imprime o contexto do estudo e chama cinco funções demonstrativas: uma para cada tipo de coesão. Essa escolha deixa a leitura mais profissional, facilita manutenção e reforça a ideia de separação de responsabilidades.

- demonstrate_communicational_cohesion();
- demonstrate_procedural_cohesion();
- demonstrate_temporal_cohesion();
- demonstrate_logical_cohesion();
- demonstrate_coincidental_cohesion();

6. Trechos essenciais do código e pontos de print

Abaixo estão os prints mais estratégicos para anexar no relatório. A ideia é manter o mesmo estilo visual da V1, mas agora mostrando os módulos da nova versão.

Trecho 1 - Cabeçalho do módulo comunicacional

```
include > modules > communicational > C employee_record.h > ...
#ifndef EMPLOYEE_RECORD_H
#define EMPLOYEE_RECORD_H

#include "models/employee.h"

float calculate_monthly_compensation(const Employee *employee);
int calculate_years_in_company(const Employee *employee, int current_year);
int has_high_attendance(const Employee *employee);
int is_eligible_for_promotion(const Employee *employee, int current_year);
void print_employee_summary(const Employee *employee, int current_year);

#endif
```

Trecho 2 - Resumo da lógica comunicacional

d M | Atividade_Coesao_em_C_Relatorio_v2.docx U | C employee_record.h U | C employee_record.c U

> modules > communicational > C employee_record.c > ...

```c
#include <stdio.h>

#include "modules/communicational/employee_record.h"

#define PROMOTION_MIN_YEARS 3
#define PROMOTION_MIN_ATTENDANCE 90.0f

/* Calcula a compensacao mensal total com base no salario e no bonus. */
float calculate_monthly_compensation(const Employee *employee) {
    return employee->base_salary + employee->monthly_bonus;
}

/* Mede ha quantos anos a funcionaria esta na empresa. */
int calculate_years_in_company(const Employee *employee, int current_year) {
    return current_year - employee->admission_year;
}

/* Verifica se a assiduidade atende ao criterio esperado. */
int has_high_attendance(const Employee *employee) {
    return employee->attendance_rate >= PROMOTION_MIN_ATTENDANCE;
```

lo\coesao_c_project\include\modules\temporal •

```c
/* Combina tempo de empresa e assiduidade para avaliar promocao. */
int is_eligible_for_promotion(const Employee *employee, int current_year) {
    const int years_in_company = calculate_years_in_company(employee, current_year);
    return years_in_company >= PROMOTION_MIN_YEARS && has_high_attendance(employee);
}

/* Todas as funcoes deste modulo giram em torno do mesmo registro: Employee. */
void print_employee_summary(const Employee *employee, int current_year) {
    printf("\n=== COESAO COMUNICACIONAL: REGISTRO DE FUNCIONARIO ===\n");
    printf("Funcionaria: %s\n", employee->name);
    printf("Cargo: %s\n", employee->role);
    printf("Salario base: R$ %.2f\n", employee->base_salary);
    printf("Bonus mensal: R$ %.2f\n", employee->monthly_bonus);
    printf("Compensacao mensal total: R$ %.2f\n", calculate_monthly_compensation(employee));
    printf("Anos na empresa: %d\n", calculate_years_in_company(employee, current_year));
    printf("Assiduidade: %.1f%%\n", employee->attendance_rate);
    printf("Elegivel para promocao: %s\n",
           is_eligible_for_promotion(employee, current_year) ? "Sim" : "Nao");
}
```

Trecho 3 - Procedimento de atendimento

```c

/* Define a prioridade do atendimento seguindo regras simples de negocio. */
static void define_priority(const ServiceTicket *ticket, TicketProcedureResult *result) {
    if (ticket->customer_waiting_days >= 7 || ticket->estimated_complexity >= 8) {
        strcpy(result->priority, "Alta");
    } else if (ticket->estimated_complexity >= 5) {
        strcpy(result->priority, "Media");
    } else {
        strcpy(result->priority, "Baixa");
    }
}

/* Estima o esforco considerando complexidade e necessidade de visita tecnica. */
static void estimate_service_hours(const ServiceTicket *ticket, TicketProcedureResult *result) {
    result->estimated_hours = ticket->estimated_complexity * 2;

    if (ticket->requires_onsite_visit) {
        result->estimated_hours += 3;
    }
}

/* Fecha o procedimento com base no que foi calculado nas etapas anteriores. */
static void define_status(const ServiceTicket *ticket, const TicketProcedureResult *result,
                          TicketProcedureResult *final_result) {
    if (ticket->customer_waiting_days > 10 && strcmp(result->priority, "Alta") == 0) {
        strcpy(final_result->status, "Escalonar imediatamente");
    } else if (final_result->estimated_hours > 12) {
        strcpy(final_result->status, "Agendar especialista");
    } else {
                                    "Atender na fila padrao");
odules\logical\notification_dispatcher.c • Untracked
}

/*
 * A coesao procedural aparece porque as funcoes fazem sentido pela ordem em que rodam.
 * Se a ordem mudar, o fluxo perde consistencia.
 */
void execute_service_procedure(const ServiceTicket *ticket, TicketProcedureResult *result) {
    define_priority(ticket, result);
    estimate_service_hours(ticket, result);
    define_status(ticket, result, result);
}

```

Trecho 4 - Tarefas de inicialização

ocx U | employee_record.h U | employee_record.c U | service_workflow.c U | startup_tasks.c U

src > modules > temporal > startup_tasks.c > ...

```c
#include <stdio.h>

#include "modules/temporal/startup_tasks.h"

/* Cada etapa abaixo existe porque precisa acontecer no momento de inicializacao. */
static void load_configuration(StartupContext *context) {
    context->configuration_loaded = 1;
}

static void initialize_logger(StartupContext *context) {
    context->logger_initialized = 1;
}

static void connect_database(StartupContext *context) {
    context->database_connected = 1;
}

static void warm_cache(StartupContext *context) {
    context->cache_warmed = 1;
}

/* Reune tarefas disparadas no mesmo instante do ciclo de vida da aplicacao. */
void run_startup_tasks(StartupContext *context) {
    load_configuration(context);
    initialize_logger(context);
    connect_database(context);
    warm_cache(context);
}

void print_startup_report(const StartupContext *context) {
    printf("\n=== COESAO TEMPORAL: TAREFAS DE INICIALIZACAO ===\n");
    printf("Servico: %s\n", context->service_name);
    printf("Ambiente: %s\n", context->environment);
    printf("Configuracao carregada: %s\n", context->configuration_loaded ? "Sim" : "Nao");
    printf("Logger iniciado: %s\n", context->logger_initialized ? "Sim" : "Nao");
    printf("Banco conectado: %s\n", context->database_connected ? "Sim" : "Nao");
    printf("Cache aquecido: %s\n", context->cache_warmed ? "Sim" : "Nao");
}
```

Trecho 5 - Dispatcher lógico de notificações

```c
static void send_email(const Notification *notification) {
    printf("E-MAIL enviado para %s: %s\n", notification->recipient, notification->message);
}

static void send_sms(const Notification *notification) {
    printf("SMS enviado para %s: %s\n", notification->recipient, notification->message);
}

static void send_push(const Notification *notification) {
    printf("PUSH enviado para %s: %s\n", notification->recipient, notification->message);
}

const char *describe_channel(NotificationChannel channel) {
    switch (channel) {
        case CHANNEL_EMAIL:
            return "E-mail";
        case CHANNEL_SMS:
            return "SMS";
        case CHANNEL_PUSH:
            return "Push";
        default:
            return "Desconhecido";
    }
}

/*
 * O modulo e logicamente coeso porque todas as opcoes pertencem ao mesmo grupo conceitual.
 * A funcao executada muda conforme o canal informado.
 */
void dispatch_notification(NotificationChannel channel, const Notification *notification) {
    printf("\n=== COESAO LOGICA: DISPARO DE NOTIFICACAO ===\n");
    printf("Canal selecionado: %s\n", describe_channel(channel));

    switch (channel) {
        case CHANNEL_EMAIL:
            send_email(notification);
            break;
        case CHANNEL_SMS:
            send_sms(notification);
            break;
        case CHANNEL_PUSH:
            send_push(notification);
            break;
        default:
            printf("Canal invalido.\n");
            break;
    }
}
```

Trecho 6 - Contraexemplo coincidental e uso no main

```c
#include <stdio.h>
#include <string.h>

#include "modules/coincidental/misc_helpers.h"

#define PI_VALUE 3.141592653589793

/*
 * Este modulo e propositalmente ruim do ponto de vista de coesao.
 * As funcoes abaixo nao compartilham um objetivo unico, nem o mesmo dado,
 * nem o mesmo momento de execucao. Elas foram agrupadas apenas por acaso.
 */
double calculate_circle_area(double radius) {
    return PI_VALUE * radius * radius;
}

int count_vowels(const char *text) {
    int total = 0;
    size_t index = 0U;

    while (text[index] != '\0') {
        const char character = (char)tolower((unsigned char)text[index]);

        if (character == 'a' || character == 'e' || character == 'i' ||
            character == 'o' || character == 'u') {
            total++;
        }

        index++;
    }

    return total;
}

int is_even(int value) {
    return value % 2 == 0;
}

void print_welcome_banner(void) {
    printf("\n=== COESAO COINCIDENTAL: UTILITARIOS MISCELANEOS ===\n");
    printf("Este modulo reune funcoes sem relacao direta entre si.\n");
    printf("Ele foi mantido apenas como contraexemplo didatico.\n");
}
```

7. Boas práticas de Clean Code adotadas na V2

- nomes de funções e estruturas orientados à intenção real do código;
- separação entre modelos, contratos de módulo e implementação;
- funções pequenas, com propósito claro e fácil leitura;
- main.c desacoplado das regras internas dos módulos;
- comentários pontuais para apoiar a explicação acadêmica sem poluir a implementação;
- exemplos didáticos humanizados, com cenários mais próximos de situações reais de sistema.

8. Conclusão

A versão 2 atende melhor ao novo enunciado da atividade porque representa exatamente os cinco tipos de coesão pedidos. O projeto continua simples de compilar, simples de executar e simples de apresentar. Além disso, a estrutura modular facilita tanto a correção do professor quanto a explicação oral dos alunos, já que cada exemplo foi construído para evidenciar um conceito específico.

Saída final do terminal após executar o programa com sucesso

Comando de compilação utilizado no Windows (PowerShell):

```
gcc -I./include ./src/main.c ./src/modules/communicational/employee_record.c ./src/modules/proced
ural/service_workflow.c  ./src/modules/temporal/startup_tasks.c  ./src/modules/logical/notification
_dispatcher.c  ./src/modules/coincidental/misc_helpers.c  -o  ./bin/coesao_tipos.exe
```

```
.\\bin\\coesao_tipos.exe
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\user\Documents\Puc\10 periodo\coesao_c_project> .\bin\coesao_tipos.exe
Complexidade: 7
Dias de espera: 9
Precisa de visita tecnica: Sim
Prioridade: Alta
Horas estimadas: 17
Decisao do procedimento: Agendar especialista

=== COESAO TEMPORAL: TAREFAS DE INICIALIZACAO ===
Servico: ClinicGateway
Ambiente: Producao
Configuracao carregada: Sim
Logger iniciado: Sim
Banco conectado: Sim
Cache aquecido: Sim

=== COESAO LOGICA: DISPARO DE NOTIFICACAO ===
Canal selecionado: E-mail
E-MAIL enviado para paciente@exemplo.com: Sua consulta foi confirmada para amanha as 14:00.

=== COESAO COINCIDENTAL: UTILITARIOS MISCELANEOS ===
Este modulo reune funcoes sem relacao direta entre si.
Ele foi mantido apenas como contraexemplo didatico.
Area do circulo de raio 3: 28.27
Quantidade de vogais em 'coesao': 4
14 e par? Sim
PS C:\Users\user\Documents\Puc\10 periodo\coesao_c_project>
```